CS361 - HW3

Student: Shrey Poshiya

Github Link: https://github.com/sposhiy33/CS361HW

> **Problem 1**
>
> You are given a min-heap H containing n numbers, an integer k, and a number x. Design an algorithm that determines whether H contains at least k numbers smaller than x, and runs in $O(k)$ time. Your answer must not depend on n.

Implementation:

Listing 1: Problem 1 (`Problem1/problem1.py`)

```python
"""
CS361 - HW3
Problem 1 - # of elements less than x
Author: Shrey Poshiya
"""

from minheap import RandomMinHeap


def lessThanX(minheap, x, k):
    """
    Use a traversal technique, we will use a queue to keep track of exploration.
    Initialize a count variable

    # the loop
    1. Traverse from the root node, first element of the minheap
    2. if root is less continue traversing to the children
       add the children of the root to the queue
        2a. increment the count by 1
    3. Dequeue element, and continue exploration from there
        3a. if one of the leaf nodes ends up being greater
            then dequeue then next child from the queue and continue
            the exploration from there


    - keep track of how many elements end up being less than k,
      return True when we hit the count == k
    - if we can do this before the queue empties out, return True
        - else if the queue empties before our count == k, return False
    """

    assert k > 0, "k must be greater than 0"

    heap = minheap.min_heap
    n = minheap.size
```

```python

    assert k < n, "k must be less than the length of the minheap"

    count = 0
    # queue the root node (starting from here)
    queue = [0]

    while len(queue) > 0:
        curr_idx = queue.pop(0)

        if heap[curr_idx] < x:
            count += 1
            if count >= k:
                return True

            # enqueue children if they exist
            left = (2 * curr_idx) + 1
            right = (2 * curr_idx) + 2
            if left < n:
                queue.append(left)
            if right < n:
                queue.append(right)

    return False


if __name__ == "__main__":

    # initialize the min heap
    minheap = RandomMinHeap(20)
    print(minheap.min_heap)

    # main code
    x = 30
    k = 5
    result = lessThanX(minheap, x, k)
    print(f"at least {k} elements less than {x}? {result}")
```

Note: the implementaton of the Min-Heap can be found in the github under the following directory: `HW3/Problem1`

> **Problem 2**
>
> Suppose you are given an undirected simple graph $G(V, E)$. Design an efficient algorithm to determine if the given graph is a tree or not. Note that you should also discuss how you want the input graph to be represented, e.g., an adjacency list or an adjacency matrix.

Implementation:

Listing 2: Problem 2 (`Problem2/problem2.py`)

```python
"""
CS361 - HW3
```

```python
Problem 2 - Is the graph a tree?
Author: Shrey Poshiya
"""

from graph import Graph
import random

def detectTree(graph) -> bool:
    """
    When traversing a tree, we should have EXACTLY
    ONE instance of a connection of the current node being
    in the visited list ... this is the parent. If more than
    one connection is in the visited list, the structure of
    the graph is not a tree.
    """

    # pick a source node (do random key)
    source = random.choice(list(graph.adj))

    # do a BFS traversal from the source
    # with an added check for a cycle
    visited = [source]
    queue = [source]

    while len(queue) > 0:

        node = queue.pop(0)

        # now do a check to see how many of its connections
        # are in the visited list, intersection
        if len(set(graph.adj[node]) & set(visited)) >= 2:
            return False

        # add the children to the queue
        for item in graph.adj[node]:
            if item not in visited:
                queue.append(item)
                visited.append(item)



    # make sure that the graph is connected
    if len(visited) == len(list(graph.adj)):
        return True
    else:
        return False


def main():

    #     0
    #   1    2
    # 3 4
    tree = Graph(5)
```

```python
    tree.add_undirected_edge(0, 1)
    tree.add_undirected_edge(0, 2)
    tree.add_undirected_edge(1, 3)
    tree.add_undirected_edge(1, 4)
    print(f"Test Case 1: Tree")
    print(f"\t Result: {detectTree(tree)}")

    # same tree from above, with a cycle
    not_tree = Graph(5)
    not_tree.add_undirected_edge(0, 1)
    not_tree.add_undirected_edge(0, 2)
    not_tree.add_undirected_edge(1, 3)
    not_tree.add_undirected_edge(1, 4)
    not_tree.add_undirected_edge(3, 0)
    print(f"Test Case 1: Tree with a Cycle ... so not tree")
    print(f"\t Result: {detectTree(not_tree)}")


if __name__ == "__main__":
    main()
```

Output:

Listing 3: Problem 2 output (`Problem2/problem2.out`)

```
Test Case 1: Tree
         Result: True
Test Case 1: Tree with a Cycle ... so not tree
         Result: False
```

Notes:

To see the specific implementation of the graph, see the following file: `HW3/Problem2/graph.py`. I opted to use a list representation for the graph as we are not explicity working with dense graphs, in which case the memory usage of the list and matrix representations would be equiavalent with the matrix having the added benfit of $O(1)$ look up time for edges. But since I am only performing a traversal, I do not need to lookup specific edges; therefore, I cannot utulize the fast lookup time.

Furthermore, my algorithm involves comparing a nodes entire adjecency set with the visited list. Since the list representation inherently stores the adjacency set, a list representation is the obvious choice (with a matrix representation, I would have to construct this set using extra operations)

> **Problem 3**
>
> Solve the Rotting Oranges problem and write the algorithm for your answer. Your solution must pass all test cases, and you must attach a screenshot of your accepted submission.

Implementation:

Listing 4: Problem 3 (`Problem3/problem3.py`)

```python
"""
CS361 - HW3
Problem 3 -- Leetcode Rotting Oranges
```

```python
Author: Shrey Poshiya
"""

from typing import List


class Solution:

    def orangesRotting(self, grid: List[List[int]]) -> int:


        # get starting positions of the twos
        twos = []
        ones = []
        for row, sublist in enumerate(grid):
            for col, element in enumerate(sublist):
                if element == 2:
                    twos.append((row, col))
                if element == 1:
                    ones.append((row, col))

        ## --- Address the trivial cases
        # if no starting rotten oranges, return null
        if (len(twos) == 0) and (len(ones) > 0):
            return -1

        # if no starting oranges at all, return 0
        if (len(twos) == 0) and (len(ones) == 0):
            return 0


        ### --- Now for the real stuff
        # prelim variables
        self.grid = grid
        self.m = len(grid)
        self.n = len(grid[0])

        self.minute = 0
        self.rotten = [item for item in twos]
        # keep track of current and next level traversals for minute count
        # 2 queues as opposed to 1 in standard BFS
        self.level_current = [item for item in twos]
        self.level_next = []

        # do BFS propogation on the current level twos
        # increment minute by 1 on each propogation
        while len(self.level_current) > 0:

            # pop the whole level
            for _ in range(len(self.level_current)):
                item = self.level_current.pop(0)
                self.BFSPropogate(item)

            if len(self.level_next) == 0:
```

```python
                break

            self.level_current = self.level_next
            self.level_next = []
            self.minute += 1

        # now that graph has been traversed, check if any oranges are still 1
        for row in grid:
            for element in row:
                if element == 1:
                    return -1

        return self.minute


    def BFSPropogate(self, source):

        fourmap = [(1,0), (0,1), (-1,0), (0,-1)]

        # in each direction, add vertex, if possible:
        for direction in fourmap:
            new_pos = tuple(a + b for a,b in zip(source, direction))

            if (0 <= new_pos[0] < self.m) and (0 <= new_pos[1] < self.n):
                if (new_pos not in self.rotten) and (self.grid[new_pos[0]][new_pos[1]] == 1):
                    self.level_next.append(new_pos)
                    self.rotten.append(new_pos)
                    self.grid[new_pos[0]][new_pos[1]] = 2


if __name__ == "__main__":

    grid1 = [[2,1,1],[1,1,0],[0,1,1]]
    print(f"grid: {grid1} --> expected 4 \n \t Result: {Solution().orangesRotting(grid1)}")

    grid2 = [[2,1,1],[0,1,1],[1,0,1]]
    print(f"grid: {grid2} --> expected -1 \n \t Result: {Solution().orangesRotting(grid2)}")

    grid3 = [[0,2]]
    print(f"grid: {grid3} --> expected 0 \n \t Result: {Solution().orangesRotting(grid3)}")
```

Output:

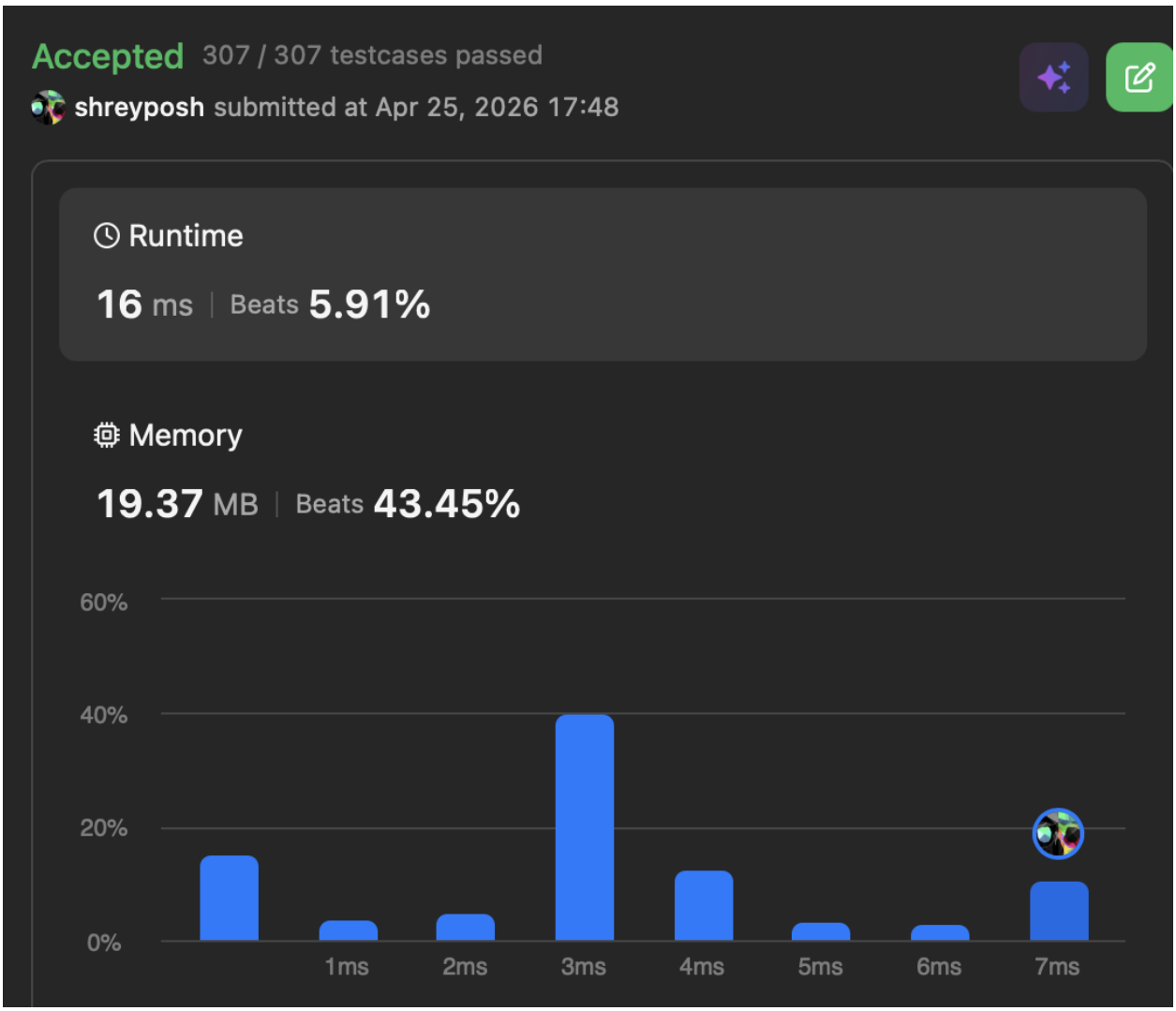
Listing 5: Problem 3 output (`Problem3/problem3.out`)

```
grid: [[2, 1, 1], [1, 1, 0], [0, 1, 1]] --> expected 4
         Result: 4
grid: [[2, 1, 1], [0, 1, 1], [1, 0, 1]] --> expected -1
         Result: -1
```

```
grid: [[0, 2]] --> expected 0
          Result: 0
```

LeetCode Submission:

Submission Link: https://leetcode.com/problems/rotting-oranges/submissions/1988162220



> **Problem 4**
>
> BFS and DFS Implementation and analysis.

Implementation:

Listing 6: Problem 4 (`Problem4/problem4.py`)

```
"""
CS361 - HW3
Problem 4 - BFS and DFS implementation
Author: Shrey Poshiya
```

```python
"""


def bfs_list(graph, source, target=None, verbose=True) -> list:
    """
    BFS for adj list representation of graph.
    Stops early when the target is reached (if given).
    Prints the length of the visited list when done.
    """

    assert source in graph.adj, "source not in graph"
    if target is not None:
        assert target in graph.adj, "target not in graph"

    visited = [source]
    queue = [source]

    found = source == target
    while len(queue) > 0 and not found:

        node = queue.pop(0)
        if verbose:
            print(f"\t pop {node} from queue")

        # add the children to the queue
        for item in graph.adj[node]:
            if verbose:
                print(f"\t   traversed node {node} -> {item}")
            if item not in visited:
                queue.append(item)
                visited.append(item)
                # early stop the moment the target is discovered
                if item == target:
                    found = True
                    break

    if verbose:
        print(f"\t BFS List visited length: {len(visited)}\n")
    return visited


def bfs_matrix(graph, source, target=None, verbose=True) -> list:
    """
    BFS for adj matrix representation.
    Stops early when the target is reached (if given).
    Prints the length of the visited list when done.
    """
    assert source in graph.idx, "source not in graph"
    if target is not None:
        assert target in graph.idx, "target not in graph"

    visited = [source]
    queue = [source]

```

```python
    found = source == target
    while len(queue) > 0 and not found:

        node = queue.pop(0)
        if verbose:
            print(f"\t pop {node} from queue")

        # scan the row to find the children
        # use row since edge may be directed -- row will work for both
        for item in graph.v:
            if graph.adj[graph.idx[node]][graph.idx[item]] == 1:
                if verbose:
                    print(f"\t   traversed node {node} -> {item}")
                if item not in visited:
                    queue.append(item)
                    visited.append(item)
                    if item == target:
                        found = True
                        break

    if verbose:
        print(f"\t BFS Matrix visited length: {len(visited)}\n")
    return visited


def dfs_list(graph, source, target=None, verbose=True) -> list:
    """
    DFS for adj list representation of graph.
    Stops early when the target is reached (if given).
    Prints the length of the visited list when done.

    visited records the actual pop order (real DFS traversal), not push
    order -- otherwise the output reads BFS-like since neighbors are
    discovered in level order even when the stack processes them deeply.
    """
    assert source in graph.adj, "source not in graph"
    if target is not None:
        assert target in graph.adj, "target not in graph"

    visited = []
    stack = [source]

    while len(stack) > 0:

        node = stack.pop()
        # node may be on the stack twice if it was a neighbor of two
        # already-popped nodes -- skip the second pop
        if node in visited:
            continue
        visited.append(node)
        if verbose:
            print(f"\t pop {node} from stack")

        if node == target:
```

```python
            break

        # push the children onto the stack
        for item in graph.adj[node]:
            if verbose:
                print(f"\t   traversed node {node} -> {item}")
            if item not in visited:
                stack.append(item)

    if verbose:
        print(f"\t DFS List visited length: {len(visited)}\n")
    return visited


def dfs_matrix(graph, source, target=None, verbose=True) -> list:
    """
    DFS for adj matrix representation.
    Stops early when the target is reached (if given).
    Prints the length of the visited list when done.

    visited records the actual pop order (real DFS traversal); see
    dfs_list for the rationale.
    """
    assert source in graph.idx, "source not in graph"
    if target is not None:
        assert target in graph.idx, "target not in graph"

    visited = []
    stack = [source]

    while len(stack) > 0:

        node = stack.pop()
        if node in visited:
            continue
        visited.append(node)
        if verbose:
            print(f"\t pop {node} from stack")

        if node == target:
            break

        # scan the row to find the children
        # use row since edge may be directed -- row will work for both
        for item in graph.v:
            if graph.adj[graph.idx[node]][graph.idx[item]] == 1:
                if verbose:
                    print(f"\t   traversed node {node} -> {item}")
                if item not in visited:
                    stack.append(item)

    if verbose:
        print(f"\t DFS Matrix visited length: {len(visited)}\n")
    return visited
```

Output:

Listing 7: Problem 4 output (`Problem4/problem4.out`)

```
=== Graph a (undirected), source 0 -> target 7 ===
Adjacency List:
         pop 0 from queue
           traversed node 0 -> 3
           traversed node 0 -> 2
           traversed node 0 -> 7
         BFS List visited length: 4

         pop 0 from stack
           traversed node 0 -> 3
           traversed node 0 -> 2
           traversed node 0 -> 7
           traversed node 0 -> 8
         pop 8 from stack
           traversed node 8 -> 0
           traversed node 8 -> 10
         pop 10 from stack
           traversed node 10 -> 8
         pop 7 from stack
         DFS List visited length: 4

Adjacency Matrix:
         pop 0 from queue
           traversed node 0 -> 2
           traversed node 0 -> 3
           traversed node 0 -> 7
         BFS Matrix visited length: 4

         pop 0 from stack
           traversed node 0 -> 2
           traversed node 0 -> 3
           traversed node 0 -> 7
           traversed node 0 -> 8
         pop 8 from stack
           traversed node 8 -> 0
           traversed node 8 -> 10
         pop 10 from stack
           traversed node 10 -> 8
         pop 7 from stack
         DFS Matrix visited length: 4


=== Graph b (directed), source A -> target O ===
Adjacency List:
         pop A from queue
           traversed node A -> B
           traversed node A -> C
         pop B from queue
           traversed node B -> D
```

```
            traversed node B -> G
          pop C from queue
            traversed node C -> D
          pop D from queue
            traversed node D -> E
          pop G from queue
            traversed node G -> H
            traversed node G -> L
            traversed node G -> N
            traversed node G -> K
          pop E from queue
            traversed node E -> F
            traversed node E -> K
          pop H from queue
            traversed node H -> I
          pop L from queue
          pop N from queue
            traversed node N -> O
          BFS List visited length: 13

          pop A from stack
            traversed node A -> B
            traversed node A -> C
          pop C from stack
            traversed node C -> D
          pop D from stack
            traversed node D -> E
          pop E from stack
            traversed node E -> F
            traversed node E -> K
          pop K from stack
            traversed node K -> L
            traversed node K -> F
          pop F from stack
            traversed node F -> G
            traversed node F -> H
            traversed node F -> M
          pop M from stack
            traversed node M -> O
          pop O from stack
          DFS List visited length: 8

Adjacency Matrix:
          pop A from queue
            traversed node A -> B
            traversed node A -> C
          pop B from queue
            traversed node B -> D
            traversed node B -> G
          pop C from queue
            traversed node C -> D
          pop D from queue
            traversed node D -> E
          pop G from queue
```

```
            traversed node G -> H
            traversed node G -> K
            traversed node G -> L
            traversed node G -> N
          pop E from queue
            traversed node E -> F
            traversed node E -> K
          pop H from queue
            traversed node H -> I
          pop K from queue
            traversed node K -> F
            traversed node K -> L
          pop L from queue
          pop N from queue
            traversed node N -> O
          BFS Matrix visited length: 13

          pop A from stack
            traversed node A -> B
            traversed node A -> C
          pop C from stack
            traversed node C -> D
          pop D from stack
            traversed node D -> E
          pop E from stack
            traversed node E -> F
            traversed node E -> K
          pop K from stack
            traversed node K -> F
            traversed node K -> L
          pop L from stack
          pop F from stack
            traversed node F -> G
            traversed node F -> H
            traversed node F -> M
          pop M from stack
            traversed node M -> O
          pop O from stack
          DFS Matrix visited length: 9


=== avg time + memory over 5 runs ===
Graph-(a):
          BFS list  : avg time =     2.38 us, avg peak memory =     176 bytes
          BFS matrix: avg time =     1.95 us, avg peak memory =     144 bytes
          DFS list  : avg time =     2.25 us, avg peak memory =     112 bytes
          DFS matrix: avg time =     3.24 us, avg peak memory =     112 bytes
Graph-(b):
          BFS list  : avg time =     5.86 us, avg peak memory =     240 bytes
```

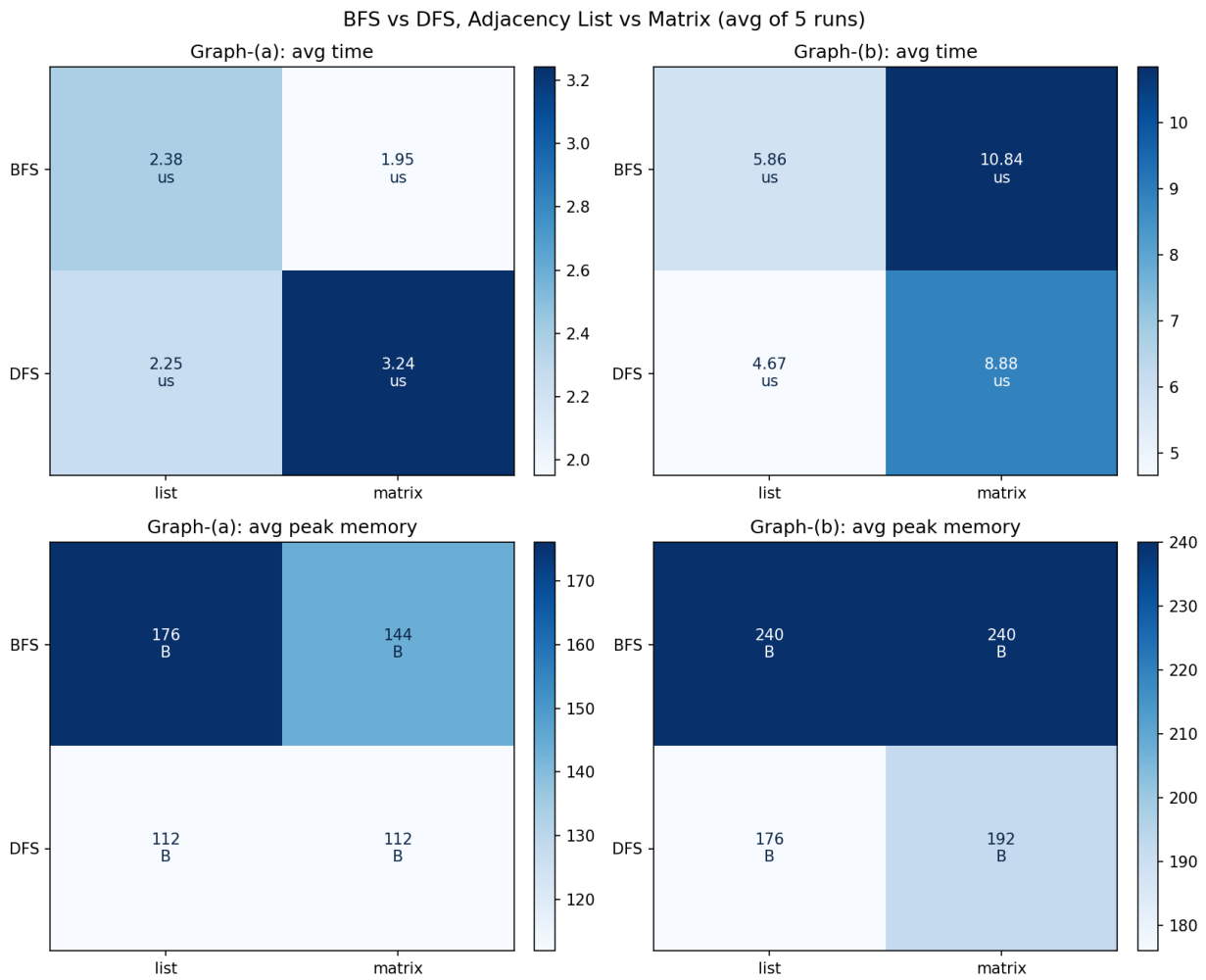
```
          BFS matrix: avg time =    10.84 us, avg peak memory =     240 bytes
          DFS list  : avg time =     4.67 us, avg peak memory =     176 bytes
          DFS matrix: avg time =     8.88 us, avg peak memory =     192 bytes

results written to results/analysis_results.csv
```

Traversal comparison (visited order until target is found):

Graph	Variant	Visited order	Length
(a)	BFS list	0, 3, 2, 7	4
(a)	BFS matrix	0, 2, 3, 7	4
(a)	DFS list	0, 8, 10, 7	4
(a)	DFS matrix	0, 8, 10, 7	4
(b)	BFS list	A, B, C, D, G, E, H, L, N, K, F, I, O	13
(b)	BFS matrix	A, B, C, D, G, E, H, K, L, N, F, I, O	13
(b)	DFS list	A, C, D, E, K, F, M, O	8
(b)	DFS matrix	A, C, D, E, K, L, F, M, O	9

Heatmap (BFS vs DFS, list vs matrix; avg of 5 runs):



Notes:

See `HW3/Problem4` for graph representation code, analysis code (time and memory analysis), and plotting code for the above heatmap. Only the main traversal algorithms are provided in the listing above.

Graph A: We see DFS ends up requiring more execution time than BFS. While the traversal counts (not the paths) are all the same, it is likely that DFS algorithm uses more operations throughout its runtime. Overall there is not as much variance between execution times due to the small size of the graph. Surprisngly, each algorithm and graph representation variant ends up have the same traversal count (this is explained later).

The max memory usage shows that BFS list representation has the largest memaory footprint out of the 4 variants. Since the traversal counts are the same, the visited arrays are all of the same size. This means that the discrepency in memory footprint must come from the queue/stack of the traversal algorithms.

Note: The peak memory metric provides insight into the peak size of the function call, which mainly correlates to peak size of the queue or stack in BFS and DFS respectively and the visited array (my implementation does not account for the memory footprint of the graph objects).

Graph B: Here we see a clear execution time increase when using a matrix representation as opposed to a list representation. This means that the time it takes to retrive all neighbors from the matrix for a given node becomes a bottleneck in the traversal algorithm. In the list representation, the specific neighbor identities for each node are inherently part of the data structure, requiring less operations than the matrix. A larger graph, shows this discrepancy more clearly as compared to the smaller graph A. The peak memory sizes correlate to the size of the visited node list (see above). This means that the queue/stack of the traversal does not have as much impact, meaning that it pretty small compared to the visited list. This goes to show that the size of queue/stack data structure in the traversal algorithm does not necessarily scale with graph size. Overall BFS has a larger memory footprint, most likely a result of the BFS traversals being longer.

Traversal Paths: It is obvious to see differences in traversal paths between DFS and BFS, but it is less obvious when comparing list vs matrix representations. The core fact underlying the difference in traversals between these two data structures is: (1) the matrix traverses over neighbors in the order of row/col order (ascending for my BFS implementation and descending for my DFS implementation) while (2) the list representation traverses over neighbors in the order that they were inserted (see analysis.py for this). Surprinsgly, despite the difference in traversal paths, we that the graph A traversals end up being the same length of 4.